THE SUMMIT PROJECTS

P1 Basic Server

Project 1 of 9 | Uses levels: Levels 0-2 | ~10 hours

1 Goal and outcome

Deploy `shop-api` **manually** on one Ubuntu server: Nginx in front, a Node.js API supervised by systemd, PostgreSQL behind it, SSH locked down, firewall on, a nightly database backup, and a script that proves it all works. No Docker, no pipeline, no cloud tooling. This is the baseline that **every later project automates, containerizes, codes or orchestrates**. If you have done it once by hand, you know what each of those tools is doing for you, and you can debug it when it does it wrong.

This project is the integration of three levels. The Level 1 capstone Capstone 1 builds the server step by step; this page tells you what "done" means for the **whole project series**, adds the pieces later projects depend on (a stable app contract, health endpoints, a backup you can restore, a verification script), and connects the server to what you learned about networking in Level 2 Capstone 2. Use the capstone for the long command listings and this page as the project brief.

You finish with:

- `curl http://<server-ip>/products` returning JSON from PostgreSQL through Nginx;
- SSH by key only, a deny-by-default firewall, and the app and database not reachable from the internet except through Nginx;
- the API as a systemd service under its own user, restarting on failure, logging to the journal;
- a nightly `pg_dump` backup and a proven restore into a scratch database;
- a `verify.sh` that prints `ALL CHECKS PASSED`, and a one-page "what I deployed" note you could hand to a colleague.

2 Architecture

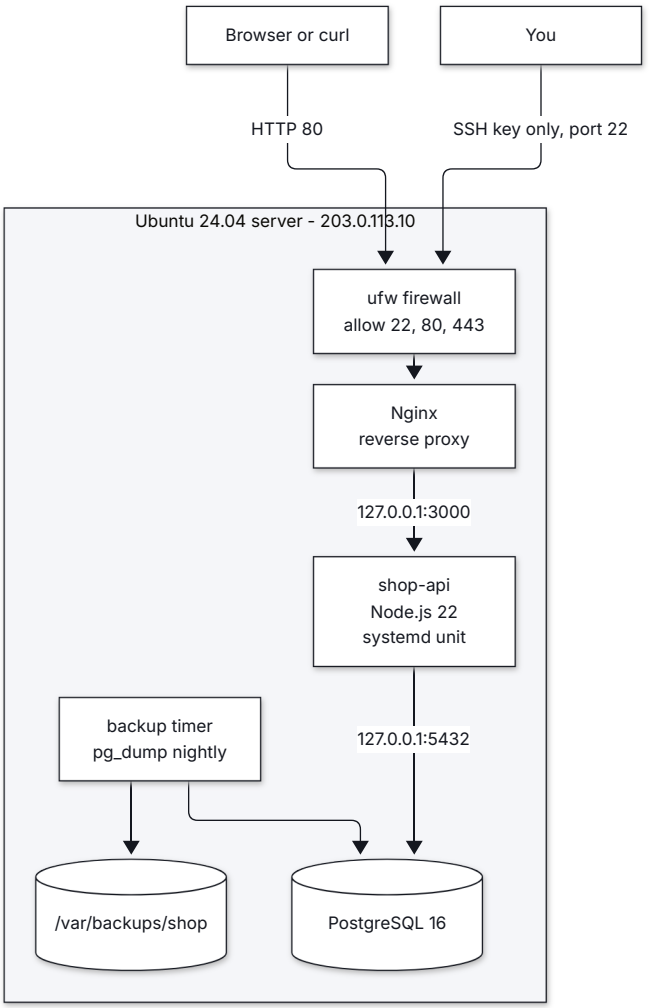


Figure 1-1

This is the starting point, so everything is new. In the next projects the boxes change but the arrows stay:

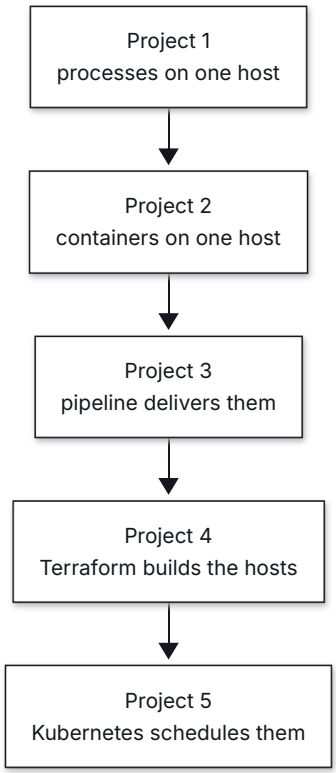


Figure 1-2

The application contract below is shared by **all nine projects**. Keep it stable; the infrastructure changes around it.

Item	Value
Listens on	`PORT` (default `3000`), bound to `127.0.0.1` here, `0.0.0.0` in containers
Configuration	environment variables only: `DATABASE_URL`, `PORT`, later `REDIS_URL`
`GET /healthz`	`200 {"status":"ok"}` as long as the process runs (liveness)
`GET /readyz`	`200` when PostgreSQL answers `SELECT 1`, otherwise `503` (readiness)
`GET /products`	JSON list from table `products`
Logs	one JSON object per line on stdout
Shutdown	on `SIGTERM`: stop accepting, drain, close the pool, exit 0

3 Prerequisites

- A blank **Ubuntu 24.04 LTS** VPS (1 vCPU / 1 GB RAM is enough; any provider; about 5 to 10 USD per month, or a local VM at no cost) and an SSH key pair. Destroy it when finished.
- Linux: filesystem hierarchy §1.1, permissions §1.6, users and sudo §1.7, systemd §1.9, journalctl §1.11, SSH keys §1.12.
- Networking: ports and firewalls §2.4, reverse proxies §2.8, network debugging tools §2.10.
- Concepts: what DevOps is Level 0, especially the SDLC §0.1 and infrastructure as code §0.3, which is what you will be doing instead of this in Project 4.
- On your laptop: `ssh`, `curl`, `git`. Replace `203.0.113.10` with your server's IP everywhere.

4 Build it step by step

Milestone 1 — Server, user and SSH hardening (≈1.5 h)

Follow Milestones M0 and M1 of the Level 1 capstone Capstone 1. The end state to reach:

```bash
# on the server, as the admin user "deploy"
sudo sshd -T | grep -Ei '^(permitrootlogin|passwordauthentication|pubkeyauthentication)'
# permitrootlogin no
# passwordauthentication no
# pubkeyauthentication yes
```

Keep a second SSH session open while you change `sshd_config`, and run `sudo sshd -t` before reloading. A typo can lock you out.

Milestone 2 — Packages: Nginx, Node.js, PostgreSQL (≈45 min)

```bash
sudo apt-get update
sudo apt-get install -y nginx postgresql ufw curl ca-certificates gnupg
# Node.js 22 LTS from NodeSource (Ubuntu's own nodejs package is older); see the L1
  capstone for the key/repo commands
node --version          # v22.x
psql --version          # psql (PostgreSQL) 16.x
nginx -v                # nginx version: nginx/1.24.x
```

Milestone 3 — Database, schema and a least-privilege role (≈45 min)

```bash
DB_PASS="$(openssl rand -base64 24 | tr -d '/+=')"
```

```
sudo -u postgres psql <<SQL
CREATE ROLE shop LOGIN PASSWORD '${DB_PASS}';
CREATE DATABASE shop OWNER shop;
SQL
sudo -u postgres psql -d shop <<'SQL'
CREATE TABLE products (
  id          serial PRIMARY KEY,
  name        text    NOT NULL,
  price_cents integer NOT NULL CHECK (price_cents ⩾ 0)
);
INSERT INTO products (name, price_cents) VALUES ('Keyboard', 4999), ('Mouse', 1999),
  ('Monitor', 17999);
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO shop;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO shop;
SQL
echo "DATABASE_URL=postgres://shop:${DB_PASS}@127.0.0.1:5432/shop" | sudo tee /etc/shop-
  api.env >/dev/null
# the env file gets its owner and mode in milestone 4, once the service account exists
ss -ltnp | grep 5432      # must show 127.0.0.1:5432 only, never 0.0.0.0
```

PostgreSQL on Ubuntu listens on loopback by default. Confirm that; do not "fix" it by opening `listen_addresses`.

Milestone 4 — The application and its systemd unit (≈1.5 h)

Create the service account, deploy directory and application (the full `server.js` is in the Level 1 capstone; add `/healthz` and `/readyz` so the contract matches):

BASH

```
sudo useradd --system --home /opt/shop-api --shell /usr/sbin/nologin shop-api
sudo install -d -o shop-api -g shop-api /opt/shop-api
sudo chown root:shop-api /etc/shop-api.env && sudo chmod 640 /etc/shop-api.env
# copy package.json, package-lock.json and server.js to /opt/shop-api, then:
sudo -u shop-api bash -c 'cd /opt/shop-api && npm ci --omit=dev --no-audit --no-fund'
```

The two health routes to add to your request handler:

JS

```
if (req.method === 'GET' && url.pathname === '/healthz') return send(res, 200, { status:
  'ok' });
if (req.method === 'GET' && url.pathname === '/readyz') {
  await pool.query('SELECT 1');                          // throws → caught below → 503
  return send(res, 200, { status: 'ready' });
}
```

`/etc/systemd/system/shop-api.service`:

INI

```
[Unit]
Description=shop-api (Node.js)
After=network-online.target postgresql.service
Wants=network-online.target

[Service]
User=shop-api
Group=shop-api
WorkingDirectory=/opt/shop-api
EnvironmentFile=/etc/shop-api.env
Environment=PORT=3000 HOST=127.0.0.1 NODE_ENV=production
ExecStart=/usr/bin/node /opt/shop-api/server.js
Restart=on-failure
RestartSec=2
# hardening: cheap, and the habit matters
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

BASH

```
sudo systemctl daemon-reload
sudo systemctl enable --now shop-api
systemctl is-active shop-api                    # active
curl -s http://127.0.0.1:3000/readyz            # {"status":"ready"}
journalctl -u shop-api -n 5 --no-pager          # JSON log lines
```

Milestone 5 — Nginx reverse proxy (≈45 min)

`/etc/nginx/sites-available/shop-api`:

NGINX

```
upstream shop_api { server 127.0.0.1:3000; keepalive 16; }

server {
    listen 80 default_server;
    server_name _;

    location / {
        proxy_pass http://shop_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 2s;
```

```
        proxy_read_timeout 15s;
    }
}
```

```bash
sudo ln -sf /etc/nginx/sites-available/shop-api /etc/nginx/sites-enabled/shop-api
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
curl -s http://127.0.0.1/products                  #
  [{"id":1,"name":"Keyboard","price_cents":4999}, ...]
```

Milestone 6 — Firewall (≈20 min)

```bash
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp          # for later; nothing listens yet
sudo ufw --force enable
sudo ufw status verbose
```

From your **laptop**, prove the exposure is what you intend: `curl -m 3 http://203.0.113.10:3000/` must time out or be refused, and `nc -vz -w 3 203.0.113.10 5432` must fail. (A closed port gives "refused", a filtered one gives a timeout. Level 2 explains why.)

Milestone 7 — Backup and verification (≈1 h)

`/usr/local/sbin/shop-backup.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
DEST=/var/backups/shop
STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
install -d -m 700 -o postgres -g postgres "$DEST"
sudo -u postgres pg_dump --format=custom --file="$DEST/shop-$STAMP.dump" shop
find "$DEST" -name 'shop-*.dump' -mtime +7 -delete       # keep a week
echo "backup ok: $DEST/shop-$STAMP.dump"
```

Run it from a systemd timer (`shop-backup.service` with `Type=oneshot` , `shop-backup.timer` with `OnCalendar=*-*-* 02:30:00` and `Persistent=true`), then restore it into a scratch database to prove it works:

```bash
sudo chmod 750 /usr/local/sbin/shop-backup.sh && sudo /usr/local/sbin/shop-backup.sh
sudo -u postgres createdb shop_restore_test
sudo -u postgres pg_restore --dbname=shop_restore_test /var/backups/shop/shop-*.dump
sudo -u postgres psql -d shop_restore_test -tAc 'SELECT count(*) FROM products;'     # 3
sudo -u postgres dropdb shop_restore_test
```

A minimal `verify.sh` (run it on the server; extend it as you go):

```bash
#!/usr/bin/env bash
fail=0
check() { if eval "$2" >/dev/null 2>&1; then echo "PASS  $1"; else echo "FAIL  $1";
  fail=1; fi; }
check "shop-api active"            "systemctl is-active --quiet shop-api"
check "nginx active"               "systemctl is-active --quiet nginx"
check "readyz through nginx"       "curl -fsS http://127.0.0.1/readyz"
check "products returns JSON"      "curl -fsS http://127.0.0.1/products | python3 -c
  'import json,sys; assert len(json.load(sys.stdin))>0'"
check "app only on loopback"       "! ss -ltn | grep -E '0\.0\.0\.0:3000|\*:3000'"
check "postgres only on loopback"  "! ss -ltn | grep -E '0\.0\.0\.0:5432|\*:5432'"
check "ssh passwords disabled"     "sudo sshd -T | grep -qi '^passwordauthentication no'"
check "ufw active"                 "sudo ufw status | grep -q 'Status: active'"
check "recent backup exists"       "sudo find /var/backups/shop -name '*.dump' -mmin -1500
  | grep -q ."
[ $fail -eq 0 ] && echo "ALL CHECKS PASSED" || { echo "SOME CHECKS FAILED"; exit 1; }
```

5 Tasks

- [ ] **T1 — Build the server from a blank image** *(≈3 h, beginner)*
  - **Goal:** a hardened host with an admin user, key-only SSH and a firewall.
  - **Steps:**
    1. Create the VPS and complete Milestone 1 and Milestone 6.
    2. From a second terminal, confirm `ssh -o PreferredAuthentications=password deploy@203.0.113.10` is rejected.
    3. Record `sudo ufw status verbose` in your notes.
  - **Done when:** key login works, password login fails with `Permission denied (publickey)` , and `ufw` shows only 22, 80, 443 allowed.

- [ ] **T2 — Get the API serving data through Nginx** *(≈2 h, beginner)*
  - **Goal:** the request path browser, Nginx, Node.js, PostgreSQL works end to end.
  - **Steps:**
    1. Complete Milestones 2 to 5.
    2. From your laptop run `curl -i http://203.0.113.10/products` and note the `Server: nginx` header.
    3. Run `sudo journalctl -u shop-api -f` while you curl, and find your request in the JSON log.
  - **Done when:** `/products` returns the three products from your laptop and the matching log line appears in the journal.
- [ ] **T3 — Prove the trust boundaries** *(≈30 min, intermediate)*
  - **Goal:** show that only Nginx is reachable.
  - **Steps:**
    1. From the laptop, probe ports 22, 80, 3000, 5432: `nc -vz -w 3 203.0.113.10 <port>`.
    2. On the server run `sudo ss -ltnp` and map each listener to a process and an address.
    3. Draw the trust boundaries on paper: which addresses can each listener be reached from?
  - **Done when:** 3000 and 5432 fail from outside, and `ss` shows them bound to `127.0.0.1`, and your drawing matches.
- [ ] **T4 — Change something safely** *(≈45 min, intermediate)*
  - **Goal:** make a code change and roll it out by hand, the way you will automate it in Project 3.
  - **Steps:**
    1. Add `GET /products/:id` (or change a price in the table).
    2. Copy the new `server.js` with `scp`, `sudo systemctl restart shop-api`, and re-test.
    3. Write the exact commands you ran into `deploy.sh` so it is repeatable (copy, restart, `curl /readyz` loop, print a failure and exit non-zero if it never becomes ready).
  - **Done when:** `./deploy.sh` redeploys and exits 0 only if `/readyz` returns 200 within 30 seconds.
- [ ] **T5 — Back up and restore** *(≈45 min, intermediate)*
  - **Goal:** a restore you have actually performed.
  - **Steps:**

1. Install the backup script and timer (Milestone 7); run `systemctl start shop-backup.service`.
2. `sudo -u postgres psql -d shop -c "DELETE FROM products WHERE id = 3;"` to simulate a mistake.
3. Restore the dump into `shop_restore_test`, and copy the missing row back into `shop`.

- **Done when:** `SELECT count(*) FROM products` in `shop` returns 3 again, and `systemctl list-timers` shows `shop-backup.timer` scheduled.

☐ **T6 — Kill it and watch it recover** *(≈30 min, intermediate)*

- **Goal:** understand supervision.
- **Steps:**
  1. `sudo systemctl kill -s SIGKILL shop-api` and watch `systemctl status shop-api` and the journal.
  2. Run `sudo systemctl stop postgresql`, then `curl -i http://127.0.0.1/readyz`, then start it again.
  3. Note which of `Restart=on-failure`, `/readyz` and Nginx's behaviour each part of this relied on.
- **Done when:** the unit restarts within about 2 seconds after SIGKILL, and `/readyz` returned 503 (not a hang) while PostgreSQL was down.

☐ **T7 — Automate your own verification** *(≈45 min, advanced)*

- **Goal:** turn "it works" into a repeatable test.
- **Steps:**
  1. Create `verify.sh` from Milestone 7 and run it with `sudo ./verify.sh`.
  2. Add two checks of your own (for example TLS expiry later, disk usage under 80 percent, `nginx -t` passing).
  3. Break one thing on purpose and confirm the script fails with a useful message.
- **Done when:** `sudo ./verify.sh` prints `ALL CHECKS PASSED` on a healthy host and a `FAIL` line naming the broken item otherwise.

☐ **T8 — Prove it: rebuild from your notes in one sitting** *(≈2 h, advanced)*

- **Goal:** show you can reproduce the host, not just that it once worked.
- **Steps:**
  1. Destroy the VPS. Create a new one. Using only your notes and `deploy.sh`, rebuild everything without reading this page.
  2. Time the rebuild and list every step you had to look up.

3. Explain aloud (or in writing) what happens from `curl` to the database row, naming each process, port and file involved.

- **Done when:** `sudo ./verify.sh` passes on the rebuilt host and you can name at least five manual steps that Projects 2 to 4 will remove.

6 Definition of done

- [ ] `curl http://<server-ip>/products` returns JSON through Nginx; `/readyz` returns 200.
- [ ] SSH password authentication and root login are disabled; the firewall allows only 22, 80 and 443.
- [ ] `shop-api` runs as the unprivileged `shop-api` user under systemd with `Restart=on-failure`; secrets are in `/etc/shop-api.env` with mode `640`.
- [ ] Ports 3000 and 5432 listen on `127.0.0.1` only and are unreachable from the internet.
- [ ] A backup timer exists, and you restored a backup into a scratch database and counted the rows.
- [ ] `verify.sh` prints `ALL CHECKS PASSED`.
- [ ] A one-page note lists the packages, files, users, ports and commands of the deployment (you are writing the first runbook).

7 Break it on purpose

Do each drill, write down the symptom you saw, the command that found the cause, and the fix.

1. **Wrong ownership on the env file.** `sudo chown root:root /etc/shop-api.env && sudo chmod 600 /etc/shop-api.env`, then restart. The service fails; find the permission error with `journalctl -u shop-api`. Fix with the correct group. *(Lesson: permissions and service accounts.)*
2. **Nginx points at the wrong port.** Change `server 127.0.0.1:3000` to `3001`, `nginx -t && systemctl reload nginx`. You get `502 Bad Gateway`; confirm in `/var/log/nginx/error.log` ("connect() failed … Connection refused"). *(Lesson: a 502 means the proxy is fine and the upstream is not.)*
3. **Break the SSH config, safely.** Keep a second SSH session open. Add the typo `PermitRootLogin noo` to a file in `/etc/ssh/sshd_config.d/`, then run `sudo sshd -t`. The validator reports the bad value before anything reloads. Remove the file. *(Lesson: validate before reload; keep a second session.)*

4. **Fill the disk.** Create a large file with `sudo fallocate -l 2G /var/tmp/filler` (size it to leave your small VPS only a little free space), watch `df -h`, and check what fails first (journal writes, PostgreSQL writes, backups). Delete the file. *(Lesson: disks are shared fate; monitoring arrives in Project 7.)*

8 How this evolves into the next project

Everything you did by hand has a cost: the server is a **pet**. Its state lives in package versions, files in `/etc`, and your memory. If it dies you rebuild it from notes, and two engineers will rebuild it differently.

Project 2 — Docker keeps the same application contract and the same request path, but puts Nginx, the API and PostgreSQL in **containers** described by a Compose file, and adds Redis as a cache. The `systemd` unit becomes a `restart:` policy and a health check, `/etc/shop-api.env` becomes environment variables, `apt install` becomes `image:`, and `verify.sh` becomes a smoke test you can run on any laptop. Keep your `deploy.sh` and `verify.sh`: you will reuse the checks and later run them from a pipeline in Project 3.